

Usando um *gerador*

Um **gerador** é o jeito curto de criar um iterador: uma função comum com `yield` no lugar de `return`. Ela entrega um valor, **pausa** e continua de onde parou no próximo pedido. Nesta aula: `yield`, a pausa entre os `next()`, o gerador como iterador pronto, as expressões geradoras e por que elas economizam memória.

`yield`

Pausa e retomada

Gerador = iterador

Expressão geradora

Memória

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Generators, Generator Expressions; Glossário: generator; Functional Programming HOWTO), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Uma função que *entrega aos poucos*

primeiro.py

```
def contagem(n):  
    while n > 0:  
        yield n          # entrega um valor e pausa aqui  
        n -= 1  
  
g = contagem(3)  
print(type(g).__name__)  
print(list(g))  
for x in contagem(2):  
    print("for:", x)
```

SAÍDA

```
generator  
[3, 2, 1]  
for: 2  
for: 1
```

- ✓ Basta um `yield` no corpo para a função virar uma **função geradora**
- ✓ Chamá-la não executa o código: devolve um objeto `generator`
- ✓ O gerador funciona em `for`, `list()`, `sum()` ... como qualquer iterador

Roda até o *próximo yield*

pausa.py

```
def passos():
    print("  começou")
    yield 1
    print("  voltou depois do 1")
    yield 2
    print("  terminou")

g = passos()          # nada roda ainda
print("criado")
print(next(g))
print(next(g))
try:
    next(g)
except StopIteration:
    print("StopIteration")
```

NEXT()

roda até um
yield



YIELD 1

entrega e pausa



NEXT()

continua dali

- ✓ "criado" aparece antes de "começou": o corpo só roda no primeiro `next()`
- ✓ As variáveis locais ficam guardadas entre uma pausa e outra
- ✓ Quando a função termina, vem o `StopIteration` automaticamente

SAÍDA

```
criado
  começou
1
  voltou depois do 1
2
  terminou
StopIteration
```

O iterador *sem a classe*

classe_vs_gerador.py

```
# Aula 01: uma classe com __iter__ e __next__
# Com gerador, o Python cria esses métodos para você:
def contagem(n):
    while n >= 1:
        yield n
        n -= 1

g = contagem(3)
print(iter(g) is g)           # é seu próprio iterador
print(hasattr(g, "__next__"))
print(list(g), list(g))      # e também se esgota
```

SAÍDA

```
True
True
[3, 2, 1] []
```

	CLASSE (AULA 01)	GERADOR
Métodos	<code>__iter__</code> e <code>__next__</code> à mão	criados pelo Python
Estado	atributos em <code>self</code>	variáveis locais
Fim	<code>raise StopIteration</code>	a função termina
Se esgota?	Sim	Sim
Na prática, gerador é o jeito preferido de criar iteradores. A classe vale quando o objeto precisa de mais métodos.		

A list comprehension *com parênteses*

expressao.py

```
quadrados = (n * n for n in range(1, 6))    # parênteses
print(quadrados)
print(list(quadrados))

# Dentro de uma função, nem precisa dos parênteses extras
print(sum(n * n for n in range(1, 6)))
print(max(len(p) for p in ["sol", "lua", "estrela"]))
```

SAÍDA

```
<generator object <genexpr> at 0x7e15ba016810>
[1, 4, 9, 16, 25]
55
7
```

- ✓ `[...]` monta a lista inteira; `(...)` cria um gerador
- ✓ Como único argumento de uma função, os parênteses da chamada bastam: `sum(n * n for n in ...)`
- ✓ Ideal quando o resultado vai direto para `sum`, `max`, `any`, `"".join` ...

Um milhão de itens *em 208 bytes*

memoria.py

```
import sys

lista = [n * 2 for n in range(1_000_000)]
gerador = (n * 2 for n in range(1_000_000))

print("lista: ", sys.getsizeof(lista), "bytes")
print("gerador:", sys.getsizeof(gerador), "bytes")
print(sum(gerador) == sum(lista))
```

SAÍDA

```
lista: 8448728 bytes
gerador: 208 bytes
True
```

- ✓ A lista guarda o milhão de números de uma vez
- ✓ O gerador guarda só a "receita" e o ponto atual: o tamanho não muda com a quantidade
- ✓ O resultado da soma é o mesmo

O preço: o gerador não tem `len()`, não aceita índice e só pode ser percorrido uma vez. Precisou disso, use lista.

Gerador de *múltiplos*

Enunciado: escreva `multiplos(numero, limite)`, um gerador que entrega os múltiplos de `numero` até `limite`.

Uma resposta possível

```
def multiplos(numero, limite):  
    """Gera os múltiplos de numero até limite."""  
    atual = numero  
    while atual <= limite:  
        yield atual  
        atual += numero  
  
print(list(multiplos(7, 50)))  
print(sum(multiplos(3, 30)))  
print(next(multiplos(9, 100)))
```

SAÍDA

```
[7, 14, 21, 28, 35, 42, 49]  
165  
9
```

- ✓ `next(multiplos(9, 100))` calcula só o primeiro valor
- ✓ Desafio: refaça com uma expressão geradora e `range`

Teste o que *ficou*

1. O que transforma uma função em geradora?

A) return

B) yield no corpo

C) async

D) Um decorador

B.

2. Ao chamar uma função geradora, o corpo:

A) Roda inteiro

B) Roda até o primeiro yield

C) Ainda não roda

D) Dá erro

C. Só no primeiro next().

3. (x for x in range(3)) é:

A) Uma tupla

B) Uma lista

C) Um gerador

D) Um set

C.

4. O que o gerador NÃO oferece?

A) next()

B) Uso em for

C) len() e índice

D) Uso em sum()

C.

Entrega, pausa *e continua*

Um `yield` transforma a função num gerador: chamar não roda nada, e cada `next()` avança até o próximo `yield`, guardando o estado. É um iterador pronto, sem classe. Com parênteses, a expressão geradora faz o mesmo em uma linha e ocupa quase nada de memória, mas só pode ser percorrida uma vez.

`yield`

`next()`

`(x for x in ...)`

Memória constante

Uma volta só

Próxima aula → 04 · Mais Geradores

Fontes: docs.python.org/3/tutorial/classes.html#generators; docs.python.org/3/tutorial/classes.html#generator-expressions; docs.python.org/3/glossary.html (generator, generator expression); docs.python.org/3/howto/functional.html#generators; docs.python.org/3/library/sys.html#sys.getsizeof, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.